
BrickRL: Reinforcement Learning for Automated Brick Placement

Swapnil Mallick

Department of Electrical and Computer Engineering
The University of British Columbia
Vancouver, BC, Canada
smallick@student.ubc.ca

Abstract

Autonomous systems like robots have been employed to automate prefabricated construction where the system is tasked to connect multiple modules together. However, constructing buildings by using unit primitives like bricks or interlocking blocks instead of prefabricated modules makes the task at hand challenging due to several reasons. Completing this task requires the agent to understand the different spatial constraints like placing the unit primitive at the correct target location or avoiding overlap of two bricks. As the task progresses, the action space and the number of invalid actions keep on increasing rapidly. In this paper, we develop a 2-D simulation environment and use it as a testbed for comparing the ability of different Reinforcement Learning algorithms to learn complete and correct brick placement policies along different target lines. We train the RL algorithms in simulation and present a comparative study to show the performance of each of the algorithms. From the results of our study, we demonstrate that learning correct and complete brick placement policies along target lines is a difficult task to accomplish. The code can be found in the following repository: <https://github.com/SwapnilMallick/BRICK-ASSEMBLY-RL-2D/tree/main>

1 Introduction

The idea of robots being able to construct complex structures using primitive objects has always been attractive to the artificial intelligence research community due to the inherent complexities associated with the task. Construction tasks can be of different types - building LEGO structures Pun et al. [2025], joining different furniture parts Ankile et al., Ankile et al. [2024], and constructing buildings Chea et al. [2020]. However, most of these tasks are challenging for robots because these tasks are contact-rich and require long-horizon planning along with high precision.

In most of these tasks, the agent needs to make a series of decisions like pick the correct object, orient it and finally place it at the target location by also maintaining the spatial constraints. Humans possess the inherent ability to understand the spatial constraints of construction tasks by either looking at the desired target structure or following the instructions required to complete the task. Not only can humans carefully plan a long-horizon task easily, but they can also spontaneously create the sequence of actions without prior practice Chung et al. [2021].

Recently, research contributions have been made to include autonomous systems like robots in construction sites to help automate the task of constructing buildings Zhu et al. [2023]. However, most of these tasks are focused on completing prefabricated construction where the autonomous system is tasked to assemble finished sections or modules of the building on site. However, constructing buildings using bricks or interlocking blocks instead of prefabricated modules makes the task even more challenging because the action space becomes enormous. Besides this, the number of invalid

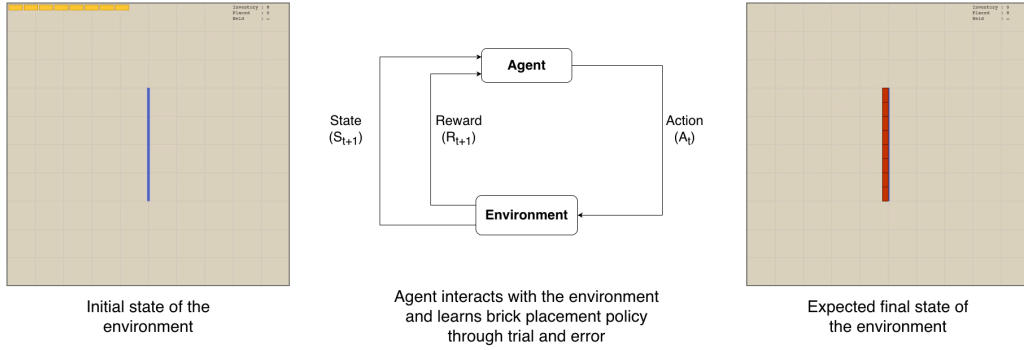


Figure 1: High-Level Overview Of The Workflow

actions in the action space keeps on growing as more bricks are placed, which makes it difficult to keep track of Chung et al. [2021].

Increasing housing prices has been a burning issue in many areas around the world. Most studies indicate that increasing demand and reduced availability are one of the main reasons for this surge in housing prices. This problem is further alleviated by the inadequate number of construction workers to build houses. The United Nations Sustainable Development Goals (SDG) address the housing issue through Target 11.1 of SDG 11, which indicates the severity of the problem. It focuses on ensuring access to adequate, safe and affordable housing by 2030 [UN-SDG, 2025].

Providing affordable housing to masses is of utmost importance for multiple reasons. The availability of affordable housing is directly related to the physical and mental well being of people. Low-income renters often have to sacrifice food, healthcare and other necessities in order to pay their rent. In addition to that, a 2023 study showed that exorbitant housing costs are the main predictors of homelessness [Heston, 2023]. Hence, automating the process of building houses could be seen as a step towards solving this problem. As housing prices rise, construction robots can reduce speed up construction times, and potentially lower overall costs.

Keeping the technical challenges and societal importance of the problem in mind, we design a 2-D simulation environment and study if standard Reinforcement Learning algorithms are able to learn policies to place bricks along simple geometric shapes denoted by a series of straight lines (Figure 1). Further details about the environment, pipeline and the RL algorithms are discussed in Section 3.

The overall contributions of this paper have been summarized below:

- We develop a simple 2-D simulation environment that is used as the testbed for realizing the potential of Reinforcement Learning algorithms to learn correct brick placement policies.
- We train standard RL algorithms and present a comparative study to show how well each algorithm fares in learning a complete and correct brick placement policy in the 2-D environment.

2 Related Works

In this section, we will discuss research contributions that try to solve similar construction problems.

Brick-by-Brick Chung et al. [2021] introduces a novel framework called *combinatorial construction* where the building agent learns to build a LEGO structure from incomplete knowledge of the target structure given in the form of 2-D images instead of volumetric information. The authors choose Reinforcement Learning to solve the problem where each state is defined as a graph of assembled bricks and their respective connections. Since the action space consists of a large number of invalid actions, an action validity prediction network is used to filter out the invalid actions for an actor-critic network.

Liu et al. [2025] solves the similar problem of combinatorial assembly by employing Deep Reinforcement Learning to learn a construction policy given the 3-D shape of the target LEGO structure. Their proposed pipeline uses an online physics-aware action mask to filter out all the invalid actions and guide policy learning. In order to ensure violation free deployment, the action with the maximum likelihood on the masked policy distribution is selected.

In Chen and Liu [2025], a two-part Deep Reinforcement Learning framework is used to make the agent learn to complete unfinished LEGO structures. At first, the incomplete structure is represented as point cloud data (PCD) and a similarity search is applied on an object library to find the matching assembled structure. Following that, a PPO Schulman et al. [2017] agent is trained to understand the unfinished structure and finish the assembly task by placing the unit primitives (e.g. LEGO blocks) sequentially.

Since freely generated LEGO structures usually do not have a plan and programming the prototyping skills for the robot is not possible for a human being without programming expertise, Liu et al. [2023a] proposes a Simulation-aided Learning from Demonstration (Sa-LfD) framework that helps deploying prototyping abilities easily to robots. At first, a human provides a demonstration of constructing a LEGO structure on the construction plate. After that, the robot generates a construction plan by extracting the task information from the human demonstration. Finally, the correctness of the learned plan is verified in simulation.

In Liu et al. [2023b], the robot masters to both assemble and disassemble LEGO structures from human demonstrations by learning the temporal task graphs. This work also proposes a digital twin to verify the correctness of the learned task before it is deployed to a FANUC LR-mate 200id/7L robot.

A framework called *InstructionNet* is proposed in Walsman et al. [2024] that tries to solve the long standing unsolved problem of understanding the structure of complex LEGO structures. The agent tries to learn building an unseen LEGO structure by gathering information about the structure and building blocks after a single interactive session as proposed by the previous *Make-and-Break* problem in LTRON Walsman et al. [2022], preparing an instruction manual using a set of images during the disassembly of an unseen structure and using this information as an explicit memory to rebuild it.

Kataoka et al. [2023] propose a system for bi-manual assembly of magnetic blocks by training a policy in simulation and transferring it to the real world. The authors create a “U-Shape Magnetic Block Assembly Task” where two robot arms have to join three magnetic blocks to make a U-shaped structure. The trained policy is found to solve the task with a success rate of $\sim 90\%$ in simulation and $\sim 50\%$ with zero-shot transfer to the real world.

In Zhu et al. [2023], the authors develop a simulator based on the Building Information Model (BIM) and train different Reinforcement Learning algorithms like DQN Mnih et al. [2013], DDQN Van Hasselt et al. [2016], PPO Schulman et al. [2017] and A2C Mnih et al. [2016] in simulation to automate assembly planning for prefabricated construction.

Apolinarska et al. [2021] proposes a Reinforcement Learning framework to make industrial robots learn to control movements to build complex timber structures that require high precision and dexterity. The control policy is trained using the Ape-X DDPG Horgan et al. [2018] algorithm, a follow-up to the well-known DDPG Lillicrap et al. [2020] algorithm, in simulation and then deployed in the real world.

A framework named APEX-MR, which is an RRT-Connect Kuffner and LaValle [2000] based sequential planner combined with Temporal Plan Graph (TPG) post-processing, is proposed in Huang et al. [2025]. The proposed pipeline is found to make bi-manual robot systems demonstrate superior coordination when deployed to perform LEGO structure assembly.

3 Model/Method

In this section, we discuss the 2-D simulation environment, the proposed pipeline and the Reinforcement Learning algorithms used to study the problem in our hand.

Simulation Environment: The designed environment is comprised of a simple 2-D grid world. The default dimension of the grid world is set to 100 x 100. The environment consists of an inventory of bricks positioned at the top-left corner of the grid world. The dimensions of each brick is set to be as 5 x 2. The target geometric shape along which the agent needs to place the bricks is represented by

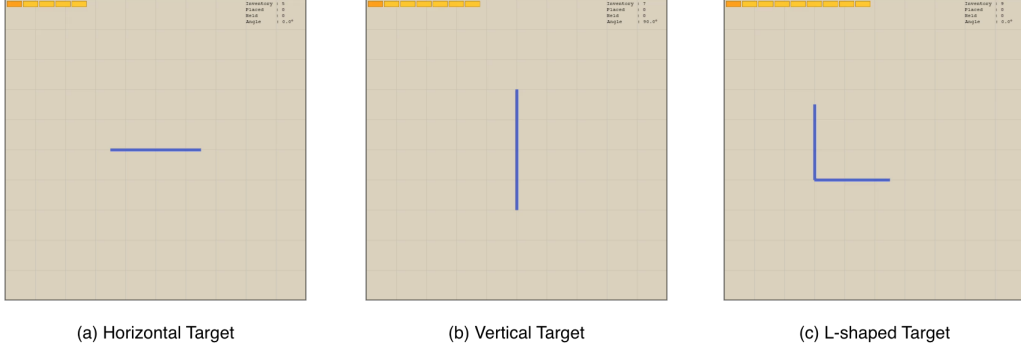


Figure 2: Initial States Of The Environment: (a) Horizontal Wall; (b) Vertical Wall; (c) L-shaped Wall

one or more straight lines. We run our experiments on three types of geometric target shapes - i) a horizontal line; ii) a vertical line; iii) an L-shaped target consisting of a horizontal and a vertical line. The total number of bricks in the inventory are six, eight and ten for the horizontal line, vertical line and L-shaped target respectively. At each timestep, the agent picks up a brick from the inventory, orients it correctly and places it along the target line. The environment checks two spatial constraints - i) whether the brick being placed is overlapping with an already placed brick or not; ii) whether the placed brick is close enough to the target line or not. The environment enforces a *proximity check* by rejecting a placement if the orthogonal projection of the brick's center falls outside the endpoints of the target line segment or the perpendicular distance of the brick from the target line segment exceeds (1.5 x brick width). Besides that, the environment also checks the *orientation alignment* by rejecting a placement if the orientation of the brick is not within 5° of the target line segment's orientation. The initial states of the environment are depicted in Figure 2, which consist of the full inventory and the target geometric shape with no brick placed along the target line(s). The final states with an empty inventory and all the bricks placed along the target geometric shape are shown in Figure 3.

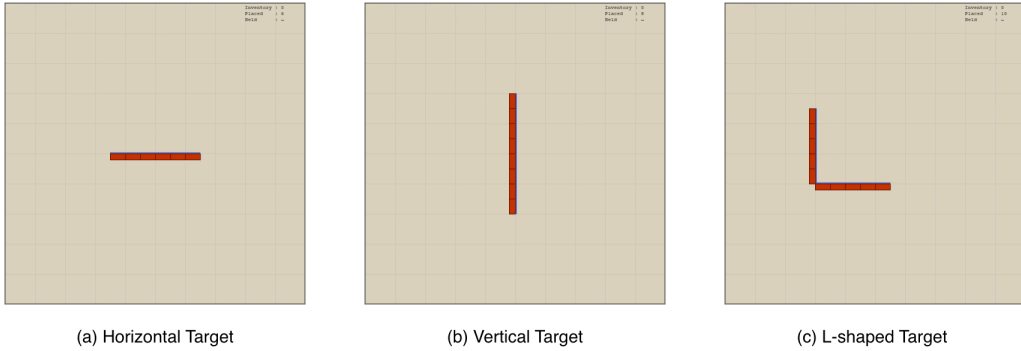


Figure 3: Final States Of The Environment: (a) Horizontal Wall; (b) Vertical Wall; (c) L-shaped Wall

Problem Formulation as an MDP: Since we have a fully observable environment, deterministic transitions and the next state following the Markov property, we can express our problem as a Markov Decision Process (MDP), $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$.

i) *State Space* ($\mathcal{S}$):

We define a state $s \in \mathcal{S}$ as a tuple:

$$s = (\{(x_i, y_i, \theta_i, \sigma_i)_{i=1}^N\}, W)$$

where:

- (x_i, y_i) refers to the position of the brick i in world coordinates
- $\theta_i \in \{0^\circ, \dots, 360^\circ\}$ refer to the orientation of the brick i
- $\sigma_i \in \{\text{IN_INVENTORY}, \text{HELD}, \text{PLACED}, \text{DISCARDED}\}$ refers to the status of brick i
- $W = \{(a_k, b_k)_{k=1}^K\}$ refers to the set of line segments in the target wall
- N refers to the total number of bricks in the inventory

ii) *Action Space ($\mathcal{A}$):*

All the three primitive actions PICK, ORIENT and PLACE are represented as single action a and is denoted by a four-element tuple, $a = (b, x_{\text{target}}, y_{\text{target}}, \theta_{\text{target}})$

where:

- b refers to the index of the brick on which the action is being executed
- x_{target} refers the x coordinate of the target location
- y_{target} refers the y coordinate of the target location
- θ_{target} refers to the target orientation

iii) *Transition Function ($\mathcal{T}$):*

For our use case, the transition function $\mathcal{T}(s, a, s')$ is deterministic in nature. Given an action a in state s , the states change in the following way:

- If status of brick i denoted by σ_i is HELD and constraint checks on the action a reflect that the target location is off the target line, then σ_i becomes DISCARDED
- If status of brick i denoted by σ_i is HELD and constraint checks on the action a reflect that the target location is on the target line but overlaps a brick that has already been placed at the same target location, then σ_i becomes DISCARDED
- If status of brick i denoted by σ_i is HELD and constraint checks on the action a reflect that the target location is on the target line without overlapping any brick that has already been placed, then σ_i becomes PLACED
- If status of brick i denoted by σ_i is IN_INVENTORY, the total number of bricks in the inventory is N and constraint checks on the action a reflect that the index brick $i \leq N$, then σ_i becomes HELD
- If the action a is attempted on a brick i with status σ_i set to PLACED or $i > N$, then σ_i remains unchanged

iv) *Reward Function ($\mathcal{R}$):*

The reward function $\mathcal{R}(s, a, s')$ depends on the result of the transition.

We define our reward signal as follows:

$$\mathcal{R}(s, a, s') = \begin{cases} +1.2 & \text{if brick is successfully placed on target line (no overlap),} \\ -0.3 & \text{if brick is placed on target line but overlapping an already placed brick,} \\ -0.8 & \text{if brick is placed completely off the target line,} \\ -2.1 & \text{if the brick picked up is invalid (i.e. already placed or index out of bounds)} \end{cases}$$

v) *Termination Condition:*

An episode either *terminates* if all the inventory bricks have been used up (i.e. either PLACED or DISCARDED) or *truncates* once a maximum of 500 steps have been reached.

vi) *Discount Factor (γ):*

Conventionally, we choose the discount factor $\gamma \in (0, 1]$.

Reinforcement Learning Algorithms: We trained three standard Reinforcement Learning algorithms from Stable-Baselines3 Raffin et al. [2021] in the 2-D simulation environment:

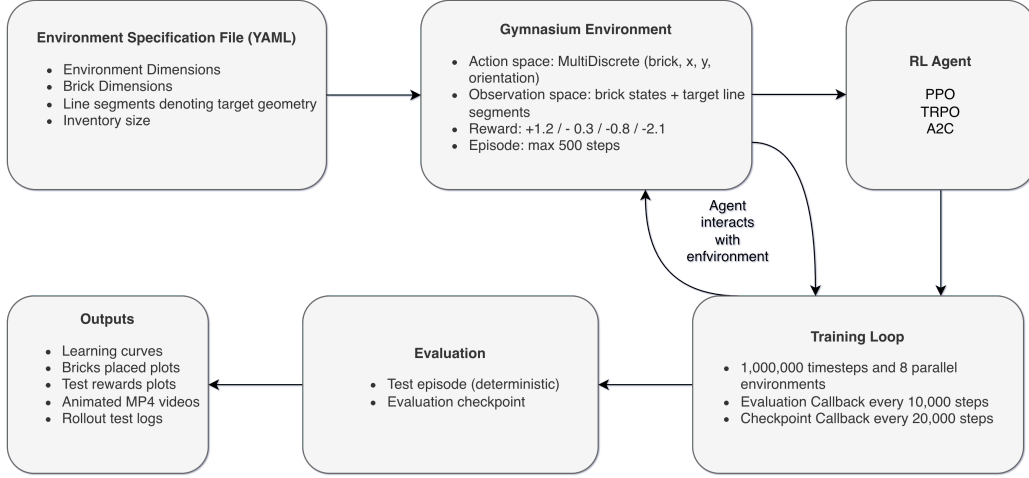


Figure 4: Training Pipeline

- Proximal Policy Optimization (PPO) Schulman et al. [2017] with Generalized Advantage Estimation (GAE) Schulman et al. [2015b]
- Trust Region Policy Optimization (TRPO) Schulman et al. [2015a], Lehmann [2024]
- Advantage Actor Critic (A2C) Mnih et al. [2016]

Training Pipeline: The training pipeline is depicted in Figure 4. A simple YAML file is used to store the information regarding the brick dimensions, the world dimensions and the target geometric shape that is represented as one or more straight lines. Since we run our experiments on three different target geometries, we have a separate YAML file for representing each of the three targets along with the world and brick dimensions. The data stored in the YAML file is the only information the system has about what the agent needs to build.

The YAML file is fed into the environment which is built on top of Gymnasium Towers et al. [2024]. At each step, the agent picks up a brick, orients it and places it at a target (x, y) location in the environment as a single joint action via a MultiDiscrete action space. Following this, the environment checks the spatial constraints to see whether the placement is valid or not and returns an appropriate reward. The aim is make the RL agent learn to place bricks at correct locations close to the target through trial and error. During training, the agent interacts with the environment, gains experience by collecting rewards through its actions and gradually updates its policy.

4 Experiments

Training Details: Each of the three RL algorithms - PPO, TRPO and A2C - is trained for one million timesteps across eight parallel environments. After training is complete, the best saved model is used in the evaluation episode, which is a single deterministic test run.

Hyper-parameters:

Hyper-parameters of the RL algorithms have been provided below:

- **PPO:** A learning rate of $3e-4$, discount factor of 0.99 and GAE coefficient of 0.95 have been used. A higher value of 0.05 has been intentionally set as the entropy coefficient instead of the default value of 0.01 so that the agent is encouraged to explore effectively. Further details have been provided in Table 3 in the Appendix.
- **TRPO:** A learning rate of $1e-3$, discount factor of 0.99 and GAE coefficient of 0.95 have been used. Further details have been provided in Table 4 in the Appendix.

- **A2C:** A learning rate of $1e-5$, discount factor of 0.99, and a GAE coefficient of 1.0 have been used. Setting the GAE coefficient to 1.0 results in n-step Monte Carlo sampling with higher variance. A2C updates more frequently ($n_steps = 16$) as compared to both PPO ($n_steps = 2048$) and TRPO ($n_steps = 2048$). Further details have been provided in Table 5 in the Appendix.

Evaluation: We compare the performance of each of the three Reinforcement Learning algorithms on the basis of four criteria -

- the learning curve;
- the total number of bricks successfully placed by each of the three algorithms on each of the three geometric targets during the test episode;
- the total test episode reward accumulated by each of the three algorithms per geometric target;
- the target wall completion rates achieved by the algorithms on each of the target wall types.

i) Learning Curves:

Figure 5 presents the learning curves of each of the three algorithms for the different target geometric shapes. From each of the learning curves, it is evident that all the algorithms begin with highly negative rewards (between ~ 20 and ~ 50) and gradually converge across all the three target geometries. This confirms that the agent learns from solely the reward feedback. PPO and A2C are found to converge faster by reaching the higher plateaus earlier. On the other hand, the learning curve of TRPO is found to rise slowly but more steadily. A thing to note about the L-shaped target is that, it shows the slowest and noisiest convergence. This reflects added difficulty of learning to place bricks along the L-shaped target owing to its geometric complexity since it consists of more than one straight lines.

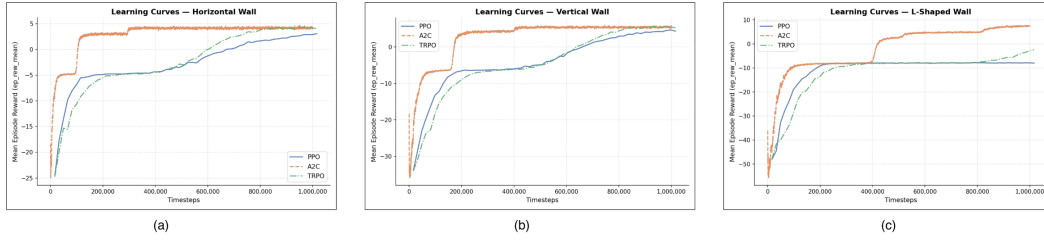


Figure 5: Learning Curves: (a) Horizontal Wall; (b) Vertical Wall; (c) L-shaped Wall

ii) Total Number of Bricks Placed Successfully:

The fourth column of Table 1 shows the total number of bricks placed by the three algorithms along each of the target lines during the deterministic test run. For the simpler targets (i.e. horizontal and vertical lines), PPO and A2C are found to be able to place a reasonable number of bricks and perform better than TRPO. For the horizontal target line, both PPO and A2C place four bricks successfully whereas TRPO manages to place three bricks. In case of the vertical target line, A2C is found to perform the best by placing six bricks successfully followed by PPO that places five bricks and TRPO that places three bricks. Even though there is not much difference in the performance of the three algorithms for the simpler geometric targets, the L-shaped target exhibits the sharpest difference in performance. A2C is able to place six bricks along the L-shaped target whereas PPO and TRPO manage to place only one brick each. The bar chart depicting the total number of bricks placed by each of the three algorithms on each of the target lines can be found in Figure 6 of Appendix A.2.

iii) Test Episode Reward:

The last column of Table 1 lists the total test episode reward collected by the three learning algorithms for each of the three targets. The reward values imitate the trend seen in the total number of bricks successfully placed by each of the algorithms.

A2C collects the highest reward on each of the three targets: +4.2 for the horizontal line, +6.6 for the vertical line and +6.0 for the L-shaped target. PPO is found to be competitive on the simpler

Table 1: Comparison of RL algorithms across different wall types.

Algorithm	Wall Type	Bricks (N)	Placed	Discarded	Total Reward
PPO	Horizontal	6	4	2	+3.200
PPO	Vertical	8	5	3	+5.100
PPO	L-shaped	10	1	9	-5.500
TRPO	Horizontal	6	3	3	+2.700
TRPO	Vertical	8	3	5	+2.100
TRPO	L-shaped	10	1	9	-2.000
A2C	Horizontal	6	4	2	+4.200
A2C	Vertical	8	6	2	+6.600
A2C	L-shaped	10	6	4	+6.000

targets, collecting a reward of +3.2 on the horizontal line and +5.1 on the vertical line. However, it's performance drops on the L-shaped target, which is evident from the accumulated reward of -5.5. TRPO is found to consistently underperform, collecting rewards of +2.7, +2.1 and -2.0 on the horizontal, vertical and L-shaped targets respectively. An interesting fact to note is that, two of the algorithms - PPO and TRPO - collect negative rewards on the L-shaped target, which shows that most of the placements made by those algorithms are invalid. The bar chart depicting the total test episode reward collected by the three learning algorithms for each of the three targets can be found in Figure 7 of Appendix A.2.

iv) *Target Wall Completion Rate:*

Table 2: Comparison of target wall completion rates across wall types.

Wall Type	PPO	TRPO	A2C
Horizontal	4/6 — 66.7%	3/6 — 50.0%	4/6 — 66.7%
Vertical	5/8 — 62.5%	3/8 — 37.5%	6/8 — 75.0%
L-shaped	1/10 — 10.0%	1/10 — 10.0%	6/10 — 60.0%
Overall	10/24 — 41.7%	7/24 — 29.2%	16/24 — 66.7%

Table 2 presents the target wall completion rates achieved by the three algorithms on each of the target wall types. Overall, A2C is found to attain the highest completion rates on each of the target wall types - 66.7% for horizontal, 75% for vertical and 60% for the L-shaped target.

The final states of the environment in each of the test episodes has been provided in Appendix A.3.

5 Conclusion and Future Work

In this paper, we designed a simple 2-D simulation environment to simulate RL agents placing bricks correctly along target lines. We train three RL algorithms - PPO, TRPO and A2C in simulation that we develop to study whether it is possible for the agent to interact with the environment and learn to place all the bricks from the inventory to the target locations along the target line. We test the performance of the algorithms on three types of geometric target shapes - i) a horizontal line; ii) a vertical line; iii) an L-shaped target. Our study shows that none of the algorithms are able to learn correct and complete brick placement policies along any of the target shapes. Overall, A2C is found to perform the best being able to place the highest number of bricks correctly along each of the three target shapes in the deterministic test run. PPO performs reasonably on the simpler targets but fails on the L-shaped target, whereas TRPO performs poorly on all the three targets.

Limitations and Future Work: The limitations of our work are as follows - i) we only run our experiments on three simple target geometric walls; ii) our approach compares the performance of three standard RL algorithms instead of proposing a novel approach to solve the problem; iii) our reward signal is purely hand-tuned, myopic and simple in nature. As an extension to this work, we would like to - i) experiment on more complex geometric targets to test the capability of the algorithms to learn correct brick placements policies; ii) design a better reward signal like potential-based reward shaping Ng et al. [1999] to guide the learning procedure; iii) come up with better learning methods like hierarchical reinforcement learning Pateria et al. [2021], Barto and Mahadevan [2003] so that the policy learns to complete the task by placing all the bricks correctly along the target.

References

- Lars Ankile, Anthony Simeonov, Idan Shenfeld, Marcel Torne, and Pulkit Agrawal. From imitation to refinement–residual rl for precise assembly, 2024. URL <https://arxiv.org/abs/2407.16677>, 11.
- Lars Ankile, Anthony Simeonov, Idan Shenfeld, and Pulkit Agrawal. Juicer: Data-efficient imitation learning for robotic assembly. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5096–5103. IEEE, 2024.
- Aleksandra Anna Apolinarska, Matteo Pacher, Hui Li, Nicholas Cote, Rafael Pastrana, Fabio Gramazio, and Matthias Kohler. Robotic assembly of timber joints using reinforcement learning. *Automation in Construction*, 125:103569, 2021.
- Andrew G Barto and Sridhar Mahadevan. Recent advances in hierarchical reinforcement learning. *Discrete event dynamic systems*, 13(4):341–379, 2003.
- Cheav Por Chea, Yu Bai, Xuebei Pan, Mehrdad Arashpour, and Yunpeng Xie. An integrated review of automation and robotic technologies for structural prefabrication and construction. *Transportation Safety and Environment*, 2(2):81–96, 2020.
- Alan Chen and Changliu Liu. Assemblycomplete: 3d combinatorial construction with deep reinforcement learning. *IFAC-PapersOnLine*, 59(30):857–862, 2025.
- Hyunsoo Chung, Jungtaek Kim, Boris Knyazev, Jinhwi Lee, Graham W Taylor, Jaesik Park, and Minsu Cho. Brick-by-brick: Combinatorial construction with deep reinforcement learning. *Advances in Neural Information Processing Systems*, 34:5745–5757, 2021.
- Thomas F Heston. The cost of living index as a primary driver of homelessness in the united states: a cross-state analysis. *Cureus*, 15(10), 2023.
- Dan Horgan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado van Hasselt, and David Silver. Distributed prioritized experience replay, 2018. URL <https://arxiv.org/abs/1803.00933>.
- Philip Huang, Ruixuan Liu, Shobhit Aggarwal, Changliu Liu, and Jiaoyang Li. Apex-mr: Multi-robot asynchronous planning and execution for cooperative assembly. *arXiv preprint arXiv:2503.15836*, 2025.
- Satoshi Kataoka, Youngseog Chung, Seyed Kamyar Seyed Ghasemipour, Pannag Sanketi, Shixiang Shane Gu, and Igor Mordatch. Bi-manual block assembly via sim-to-real reinforcement learning. *arXiv preprint arXiv:2303.14870*, 2023.
- James J Kuffner and Steven M LaValle. Rrt-connect: An efficient approach to single-query path planning. In *Proceedings 2000 ICRA. Millennium conference. IEEE international conference on robotics and automation. Symposia proceedings (Cat. No. 00CH37065)*, volume 2, pages 995–1001. IEEE, 2000.
- Matthias Lehmann. The definitive guide to policy gradients in deep reinforcement learning: Theory, algorithms and implementations. *arXiv preprint arXiv:2401.13662*, 2024.
- Timothy Paul Lillicrap, Jonathan James Hunt, Alexander Pritzel, Nicolas Manfred Otto Heess, Tom Erez, Yuval Tassa, David Silver, and Daniel Pieter Wierstra. Continuous control with deep reinforcement learning, September 15 2020. US Patent 10,776,692.
- Ruixuan Liu, Alan Chen, Xusheng Luo, and Changliu Liu. Simulation-aided learning from demonstration for robotic lego construction. *arXiv preprint arXiv:2309.11010*, 2023a.
- Ruixuan Liu, Yifan Sun, and Changliu Liu. Robotic lego assembly and disassembly from human demonstration. *arXiv preprint arXiv:2305.15667*, 2023b.
- Ruixuan Liu, Alan Chen, Weiye Zhao, and Changliu Liu. Physics-aware combinatorial assembly sequence planning using data-free action masking. *IEEE Robotics and Automation Letters*, 2025.

- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PmLR, 2016.
- Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning, ICML ’99*, page 278–287, San Francisco, CA, USA, 1999. Morgan Kaufmann Publishers Inc. ISBN 1558606122.
- Shubham Pateria, Budhitama Subagdja, Ah-hwee Tan, and Chai Quek. Hierarchical reinforcement learning: A comprehensive survey. *ACM Computing Surveys (CSUR)*, 54(5):1–35, 2021.
- Ava Pun, Kangle Deng, Ruixuan Liu, Deva Ramanan, Changliu Liu, and Jun-Yan Zhu. Generating physically stable and buildable brick structures from text. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 14798–14809, 2025.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015a.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015b.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- UN-SDG. Sustainable cities and human settlements, 2025. URL <https://sdgs.un.org/topics/sustainable-cities-and-human-settlements>.
- Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.
- Aaron Walsman, Muru Zhang, Klemen Kotar, Karthik Desingh, Ali Farhadi, and Dieter Fox. Break and make: Interactive structural understanding using lego bricks. In *European Conference on Computer Vision*, pages 90–107. Springer, 2022.
- Aaron Walsman, Muru Zhang, Adam Fishman, Ali Farhadi, and Dieter Fox. Learning to build by building your own instructions. In *European Conference on Computer Vision*, pages 261–278. Springer, 2024.
- Aiyu Zhu, Tianhong Dai, Gangyan Xu, Pieter Pauwels, Bauke De Vries, and Meng Fang. Deep reinforcement learning for real-time assembly planning in robot-based prefabricated construction. *IEEE Transactions on Automation Science and Engineering*, 20(3):1515–1526, 2023.

A Appendix:

A.1 Hyper-parameters:

A.1.1 PPO Hyper-parameters:

Table 3: PPO Hyperparameters

Hyperparameter	Value	Description
policy	MlpPolicy	Multi-layer perceptron policy network
optimizer	Adam	Optimizer for policy and value network updates
learning_rate	3e-4	Step size for Adam optimizer
n_steps	2048	Steps collected per environment per rollout
batch_size	64	Minibatch size for gradient updates
n_epochs	10	Passes over the rollout buffer per update
gamma	0.99	Discount factor for future rewards
gae_lambda	0.95	GAE smoothing parameter
clip_range	0.2	PPO clipping parameter for the policy ratio
ent_coef	0.05	Entropy bonus — set high to encourage exploration
seed	42	Random seed for reproducibility
n_envs	4	Parallel training environments
total_timesteps	500,000	Total environment interaction steps
eval_freq	10,000	Evaluation frequency (in total steps)
n_eval_episodes	10	Episodes per evaluation run
checkpoint_freq	20,000	Steps between model checkpoint saves

A.1.2 TRPO Hyper-parameters:

Table 4: TRPO Hyperparameters

Hyperparameter	Value	Description
policy	MlpPolicy	Multi-layer perceptron policy network
actor_optimizer	Natural Gradient (CG + line search)	KL-constrained update — no gradient descent for the actor
critic_optimizer	Adam	Optimizer for the value function
learning_rate	1e-3	Adam learning rate for the critic (value function)
n_steps	2048	Steps collected per environment per rollout
n_critic_updates	10	Gradient steps on the critic per rollout
target_kl	0.01	Hard KL divergence constraint per policy update
gamma	0.99	Discount factor for future rewards
gae_lambda	0.95	GAE smoothing parameter (bias-variance tradeoff)
cg_max_steps	15	Maximum conjugate gradient solver iterations
cg_damping	0.1	Damping factor for the Fisher-vector product
line_search_max_iter	10	Maximum backtracking steps in the line search
seed	42	Random seed for reproducibility
n_envs	4	Parallel training environments
total_timesteps	500,000	Total environment interaction steps
eval_freq	10,000	Evaluation frequency (in total steps)
n_eval_episodes	10	Episodes per evaluation run
checkpoint_freq	20,000	Steps between model checkpoint saves

A.1.3 A2C Hyper-parameters:

Table 5: A2C Hyperparameters

Hyperparameter	Value	Description
policy	MlpPolicy	Multi-layer perceptron policy network
optimizer	RMSProp	Optimizer for both actor and critic updates
learning_rate	7e-4	RMSProp learning rate
rms_prop_eps	1e-5	Epsilon for numerical stability in RMSProp
n_steps	16	Steps collected per environment before each update
gamma	0.99	Discount factor for future rewards
gae_lambda	1.0	GAE parameter — set to 1.0 for pure n-step returns (no bias correction)
ent_coef	0.05	Entropy bonus coefficient to encourage exploration
vf_coef	0.5	Value function loss weight in the combined loss
max_grad_norm	0.5	Gradient clipping threshold (L2 norm)
use_rms_prop	True	Use RMSProp instead of Adam
seed	42	Random seed for reproducibility
n_envs	4	Parallel training environments
total_timesteps	500,000	Total environment interaction steps
eval_freq	10,000	Evaluation frequency (in total steps)
n_eval_episodes	10	Episodes per evaluation run
checkpoint_freq	20,000	Steps between model checkpoint saves

A.2 Additional Plots:

A.2.1 Total Number of Bricks Placed Successfully:

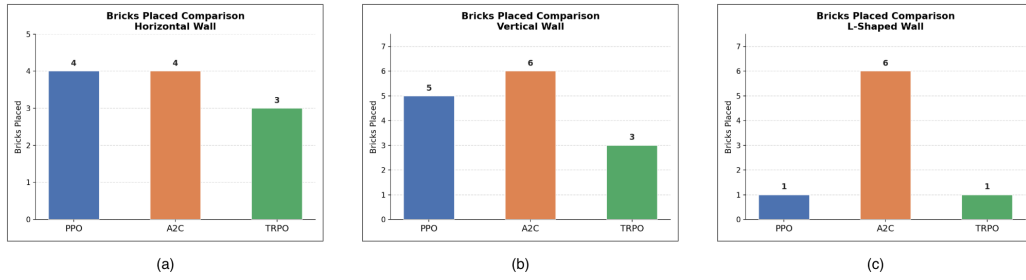


Figure 6: Total Number of Bricks Placed Successfully: (a) Horizontal Wall; (b) Vertical Wall; (c) L-shaped Wall

A.2.2 Test Episode Reward:

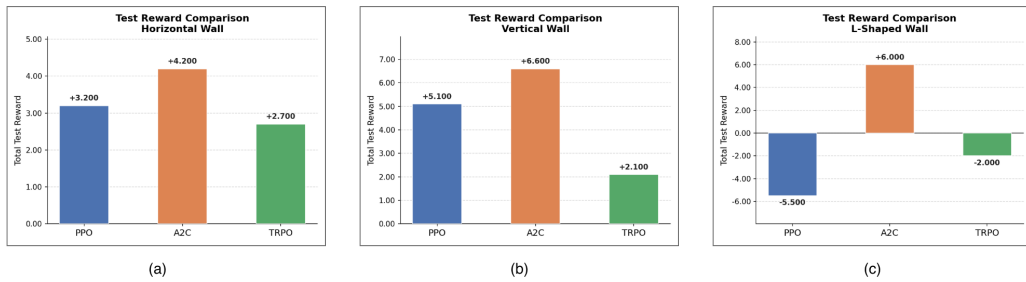


Figure 7: Test Episode Reward Accumulated: (a) Horizontal Wall; (b) Vertical Wall; (c) L-shaped Wall

A.3 Final States Of The Environment In Each Of The Test Episodes:

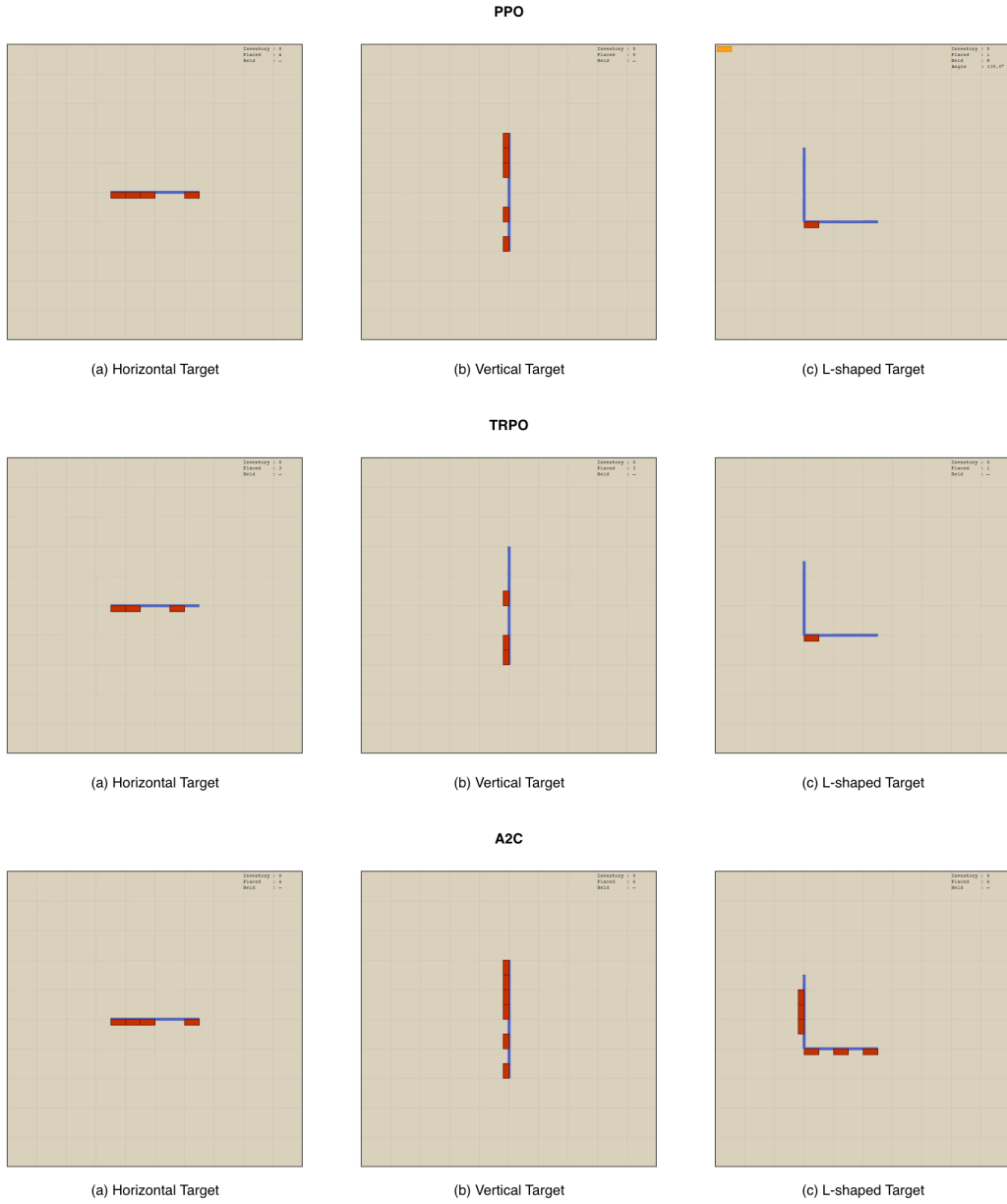


Figure 8: Final States Of The Environment In Each Of The Test Episodes

The recorded videos of the test episodes can be found at: <https://github.com/SwapnilMallick/BRICK-ASSEMBLY-RL-2D/tree/main/videos>